

OS creates a process and initializes the PCB (assigns PID, sets initial state)
OS allocates memory in RAM for code, stack, heap, and data
OS loads program code into the text segment and data into the data segment
OS may use demand paging instead of loading the entire program at once
OS initializes execution by setting the program counter and CPU registers
OS places the process in the ready queue (state = READY)
CPU scheduler selects the process and execution begins (state = RUNNING)

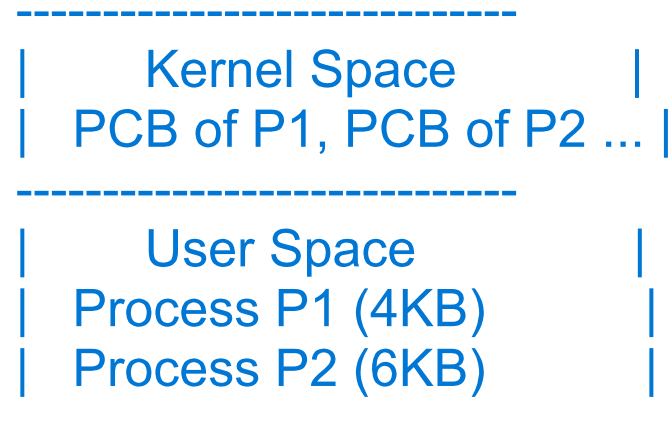
Memory Management

1. CPU is connected to RAM
2. RAM acts as a bridge
3. Secondary memory is accessed indirectly via RAM
 1. Program is stored in secondary memory (disk)
 2. OS loads it into RAM
 3. CPU executes it from RAM

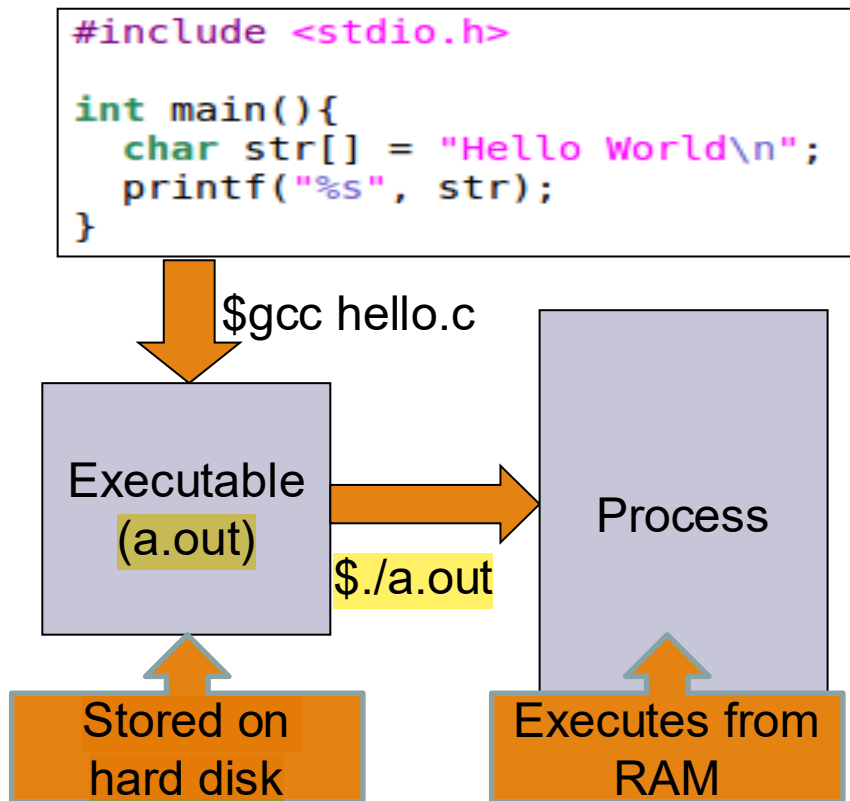
Outline

- Background
- Swapping
- Contiguous Memory Allocation
- Segmentation
- Paging
- Structure of the Page Table
- Segmentation, Case study on UNIX OS

RAM



Executing Programs (Process)

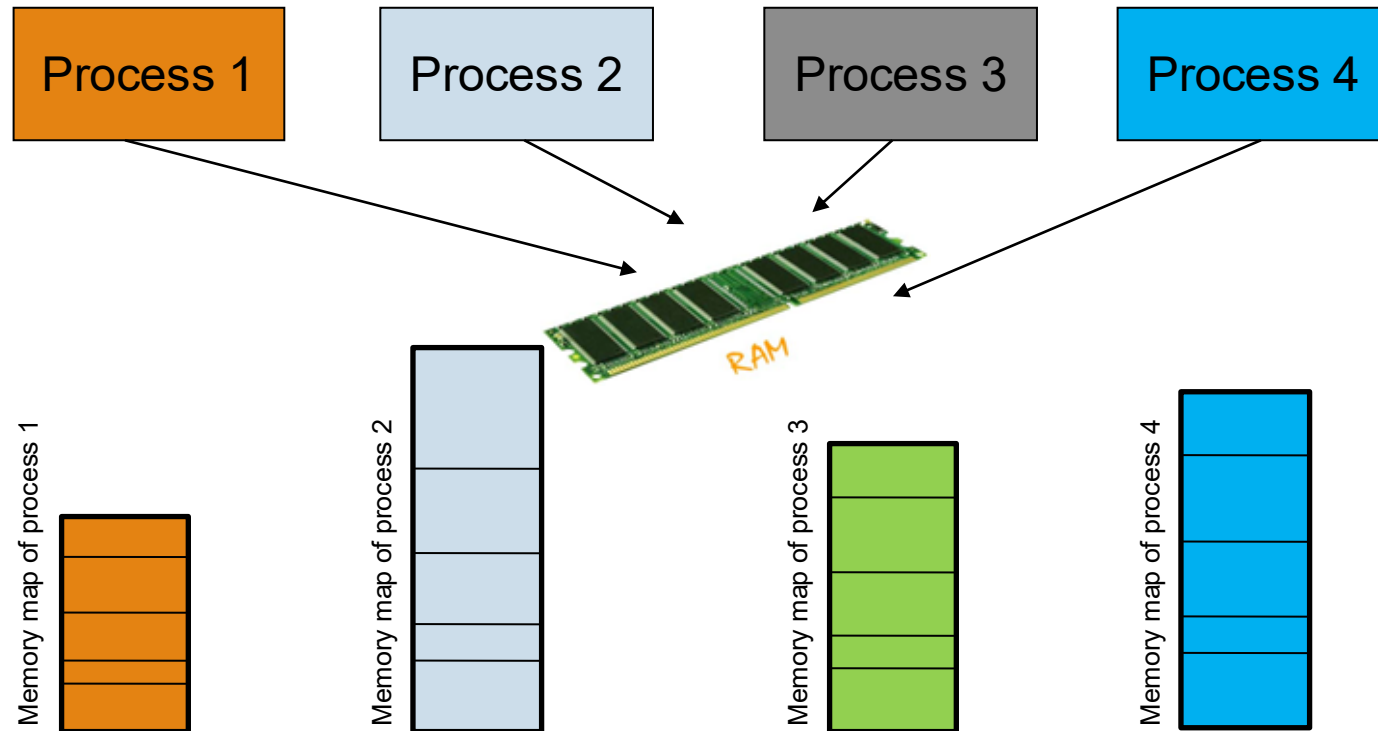


- **Process**

- A program in execution
- Present in the RAM
- Comprises of
 - Executable instructions
 - Stack
 - Heap
 - State in the OS (in kernel)
- State contains : registers, list of open files, related processes, etc.

Program (disk) → system call → OS loads into RAM → PCB created
→ process ready → CPU executes

Sharing RAM



Registers: Inside CPU, Directly accessed, Ultra-fast

Main Memory (RAM): Outside CPU, Takes more time → CPU may wait

Cache: Small, fast memory between CPU and RAM, Stores frequently used data, thus Reduces delay

Background

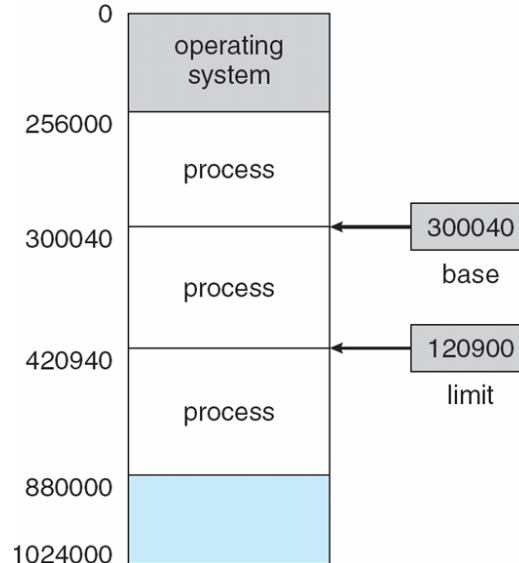
- Program must be brought (from disk) into memory and placed within a process for it to be run
- Main memory and registers are only storage CPU can access directly
- Memory unit only sees a stream of addresses + read requests, or address + data and write requests
- Register access in one CPU clock (or less)
- Main memory can take many cycles, causing a **stall**
- **Cache** sits between main memory and CPU registers
- Protection of memory required to ensure correct operation

hardware method of address protection

Background...

A pair of **base** and **limit registers** define the logical address space

CPU must check every memory access generated in user mode to be sure it is between base and limit for that user



Base Register:
Starting address of
process in RAM

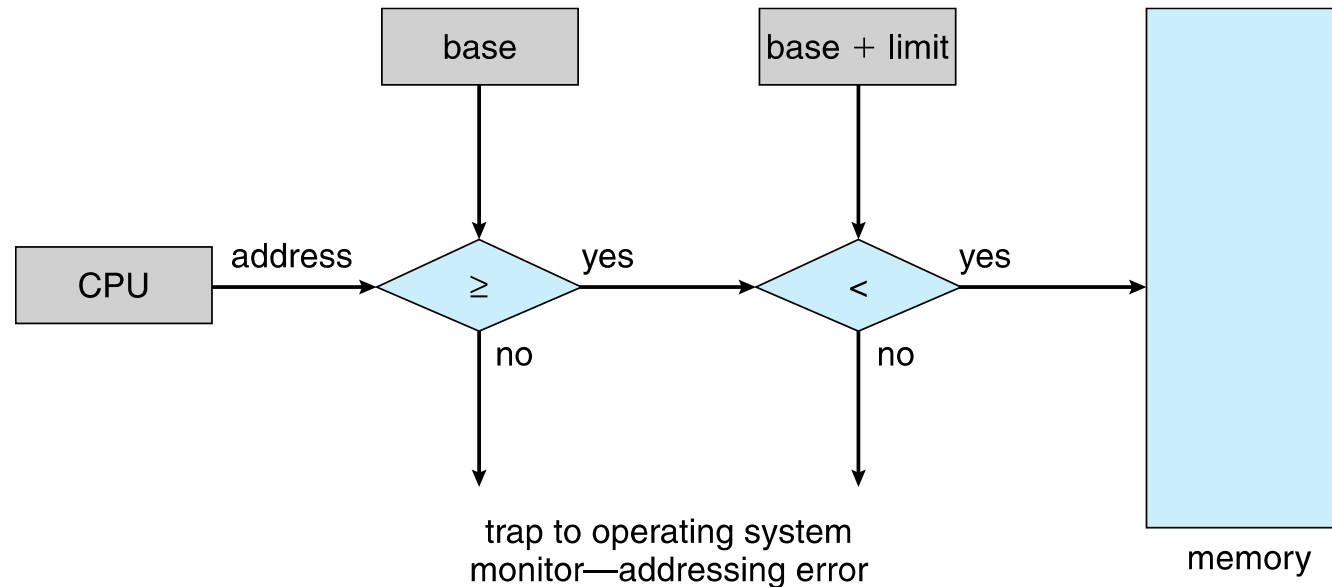
Limit Register:
Size (range) of process
memory

CPU GIVES PHYSICAL ADDRESS:

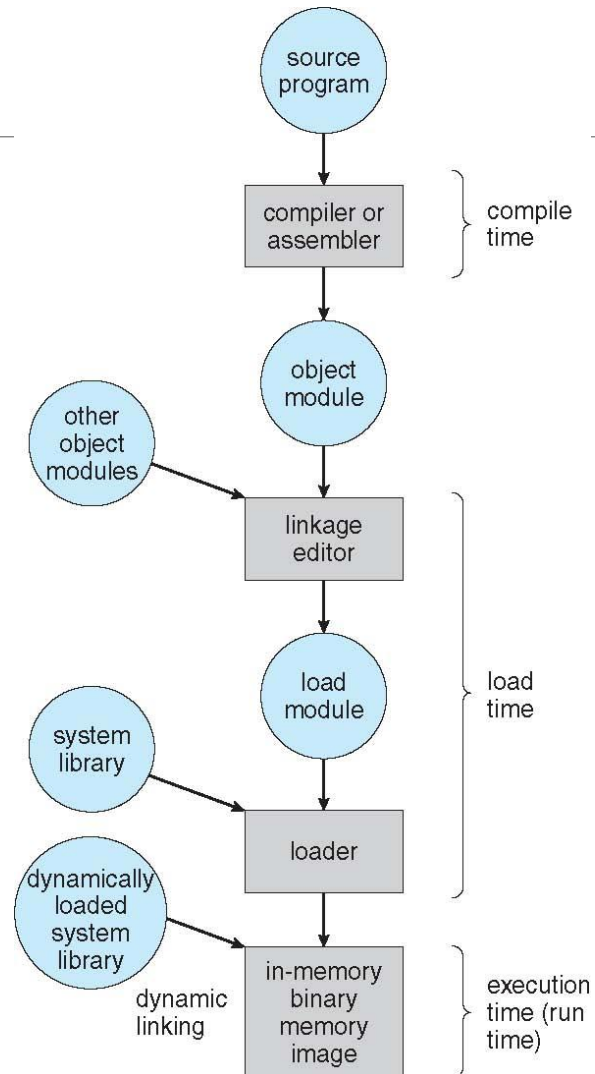
Logical Address < Limit

Physical Address = Base + Logical Address

Hardware Address Protection



Multistep Processing of a User Program



Address Binding

- Programs on disk, ready to be brought into memory to execute form an **input queue**
 - Without support, must be loaded into address 0000
- Inconvenient to have first user process physical address always at 0000
- Further, addresses represented in different ways at different stages of a program's life
 - Source code addresses usually symbolic
 - Compiled code addresses **bind** to relocatable addresses
i.e. “14 bytes from beginning of this module”
 - Linker or loader will bind relocatable addresses to absolute addresses
i.e. 74014
 - Each binding maps one address space to another
 - **Compile Time, Load time, Execution time**

Logical vs. Physical Address Space

The concept of a logical address space that is bound to a separate **physical address space** is central to proper memory management

- **Logical address** – generated by the CPU; also referred to as **virtual address**
- **Physical address** – address seen by the memory unit

Logical and physical addresses are the same in compile-time and load-time address-binding schemes; logical (virtual) and physical addresses differ in execution-time address-binding scheme

Logical address space is the set of all logical addresses generated by a program

Physical address space is the set of all physical addresses generated by a program

Dynamic Loading

- Routine is not loaded until it is called
- Better memory-space utilization; unused routine is never loaded
- All routines kept on disk in relocatable load format
- Useful when large amounts of code are needed to handle infrequently occurring cases
- No special support from the operating system is required
 - Implemented through program design
 - OS can help by providing libraries to implement dynamic loading

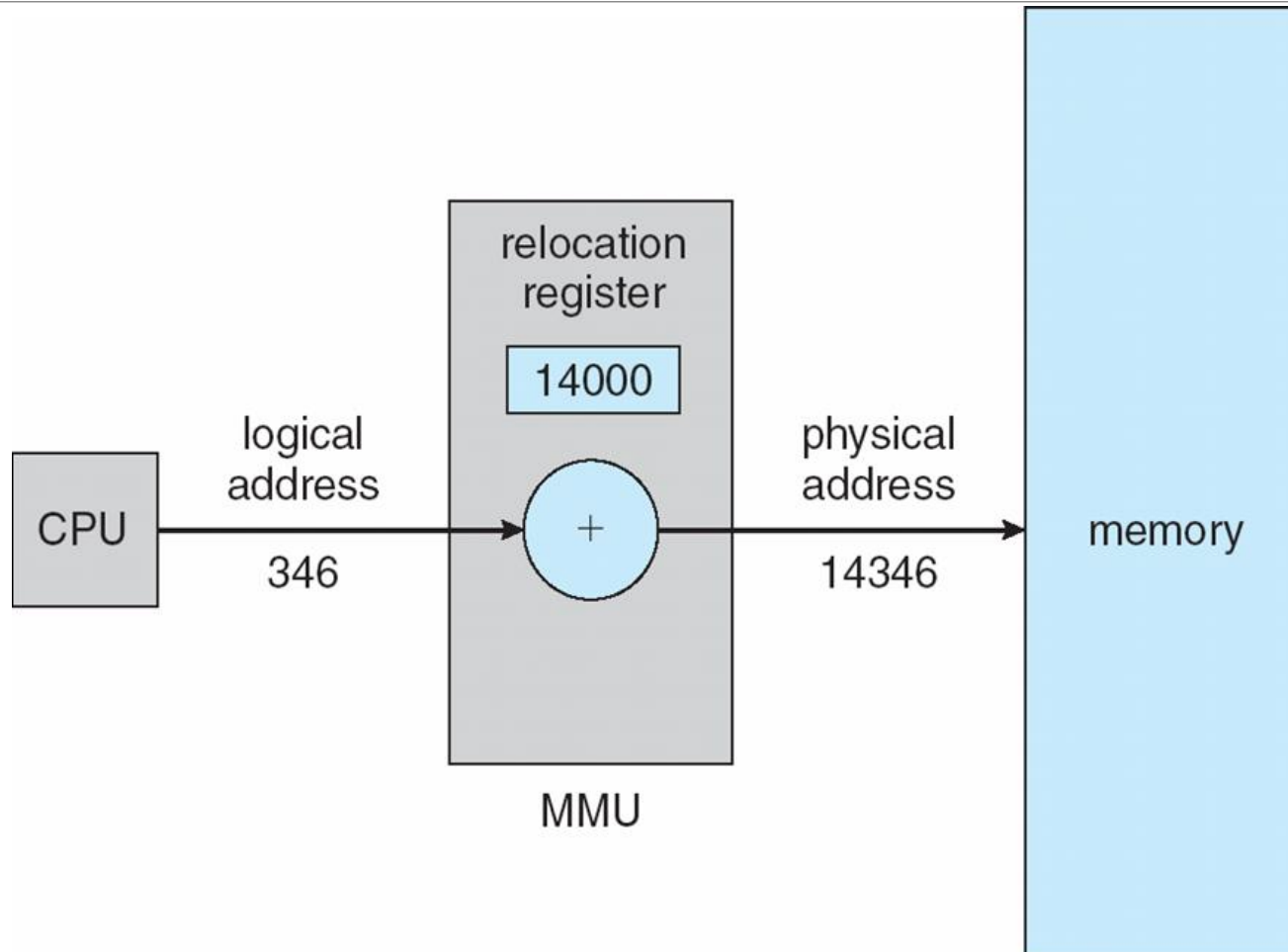
Dynamic Linking

- **Static linking** – system libraries and program code combined by the loader into the binary program image
- Dynamic linking – linking postponed until execution time
- Small piece of code, **stub**, used to locate the appropriate memory-resident library routine
- Stub replaces itself with the address of the routine, and executes the routine
- Operating system checks if routine is in processes' memory address
 - If not in address space, add to address space
- Dynamic linking is particularly useful for libraries
- System also known as **shared libraries**

Memory-Management Unit (MMU)

- Hardware device that at run time maps virtual to physical address
- Many methods possible, covered in the rest of this chapter
- To start, consider simple scheme where the value in the relocation register is added to every address generated by a user process at the time it is sent to memory
 - Base register now called **relocation register**
- The user program deals with *logical* addresses; it never sees the *real* physical addresses
 - Execution-time binding occurs when reference is made to location in memory
 - Logical address bound to physical addresses

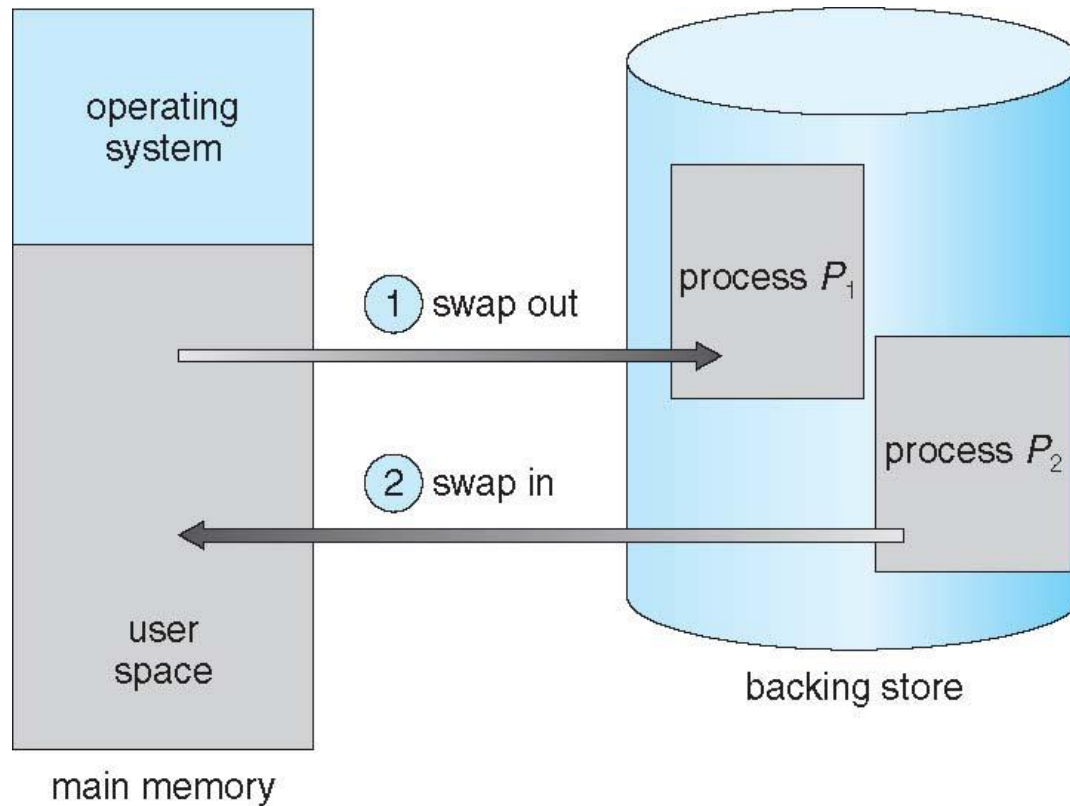
Dynamic relocation using a relocation register...



Swapping

- A process can be **swapped** temporarily out of memory to a backing store, and then brought back into memory for continued execution
 - Total physical memory space of processes can exceed physical memory
- **Backing store** – fast disk large enough to accommodate copies of all memory images for all users; must provide direct access to these memory images
- **Roll out, roll in** – swapping variant used for priority-based scheduling algorithms; lower-priority process is swapped out so higher-priority process can be loaded and executed
- Major part of swap time is transfer time; total transfer time is directly proportional to the amount of memory swapped
- System maintains a **ready queue** of ready-to-run processes which have memory images on disk

Schematic View of Swapping



Swapping ...

- Does the swapped out process need to swap back into same physical addresses?
 - Depends on address binding method
- Modified versions of swapping are found on many systems (i.e., UNIX, Linux, and Windows)
 - Swapping normally disabled
 - Started if more than threshold amount of memory allocated
 - Disabled again once memory demand reduced below threshold

Context Switch Time including Swapping

- If next processes to be put on CPU is not in memory, need to swap out a process and swap in target process
- Context switch time can then be very high
- 100MB process swapping to hard disk with transfer rate of 50MB/sec
 - Swap out time of 2000 ms
 - Plus swap in of same sized process
 - Total context switch swapping component time of 4000ms (4 seconds)
- Can reduce if reduce size of memory swapped – by knowing how much memory really being used
 - System calls to inform OS of memory use via `request_memory()` and `release_memory()`

Contiguous Allocation

- Main memory must support both OS and user processes
- Limited resource, must allocate efficiently
- Contiguous allocation is one early method
- Main memory usually into two **partitions**:
 - Resident operating system, usually held in low memory with interrupt vector
 - User processes then held in high memory
 - Each process contained in single contiguous section of memory

Contiguous Allocation ...

- Relocation registers used to protect user processes from each other, and from changing operating-system code and data
 - Base register contains value of smallest physical address
 - Limit register contains range of logical addresses – each logical address must be less than the limit register
 - MMU maps logical address *dynamically*

Can then allow actions such as kernel code being **transient** and kernel changing size

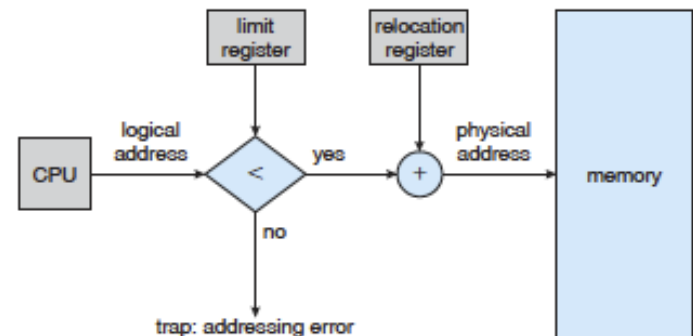


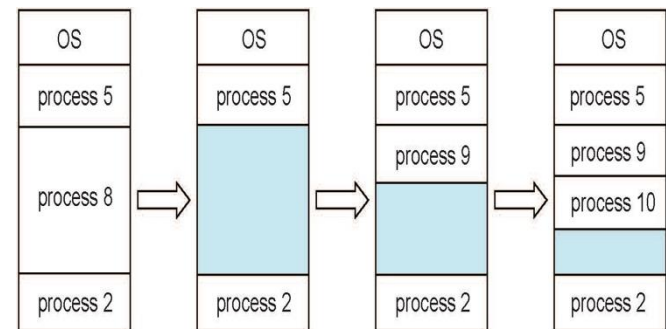
Figure 7.6 Hardware support for relocation and limit registers.

Contiguous Allocation...

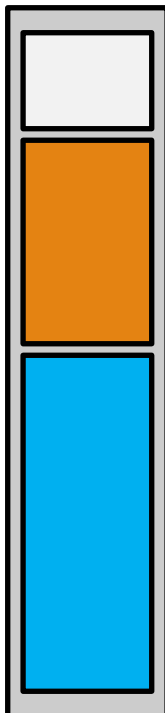
- Multiple-partition allocation
 - Degree of multiprogramming limited by number of partitions
 - Variable-partition sizes for efficiency (sized to a given process' needs)
 - Hole – block of available memory; holes of various size are scattered throughout memory
 - When a process arrives, it is allocated memory from a hole large enough to accommodate it
 - Process exiting frees its partition, **adjacent free partitions combined**
 - Operating system maintains information about:
 - a) allocated partitions
 - b) free partitions (hole)

Partition Table

Memory Address	Size	Process	Usage
0x0	120k	4	In use
120k	60k	1	In use
180k	30k		Free



RAM



Contiguous Allocation...

As long sufficient contiguous space is available, new processes allocated memory.

Process 5 of size 20k started

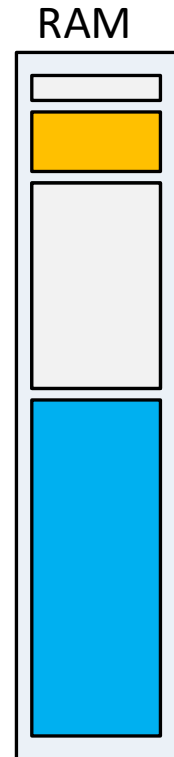
Process 5 allocated RAM

Process 1 completed

Process 1 deallocated
from RAM

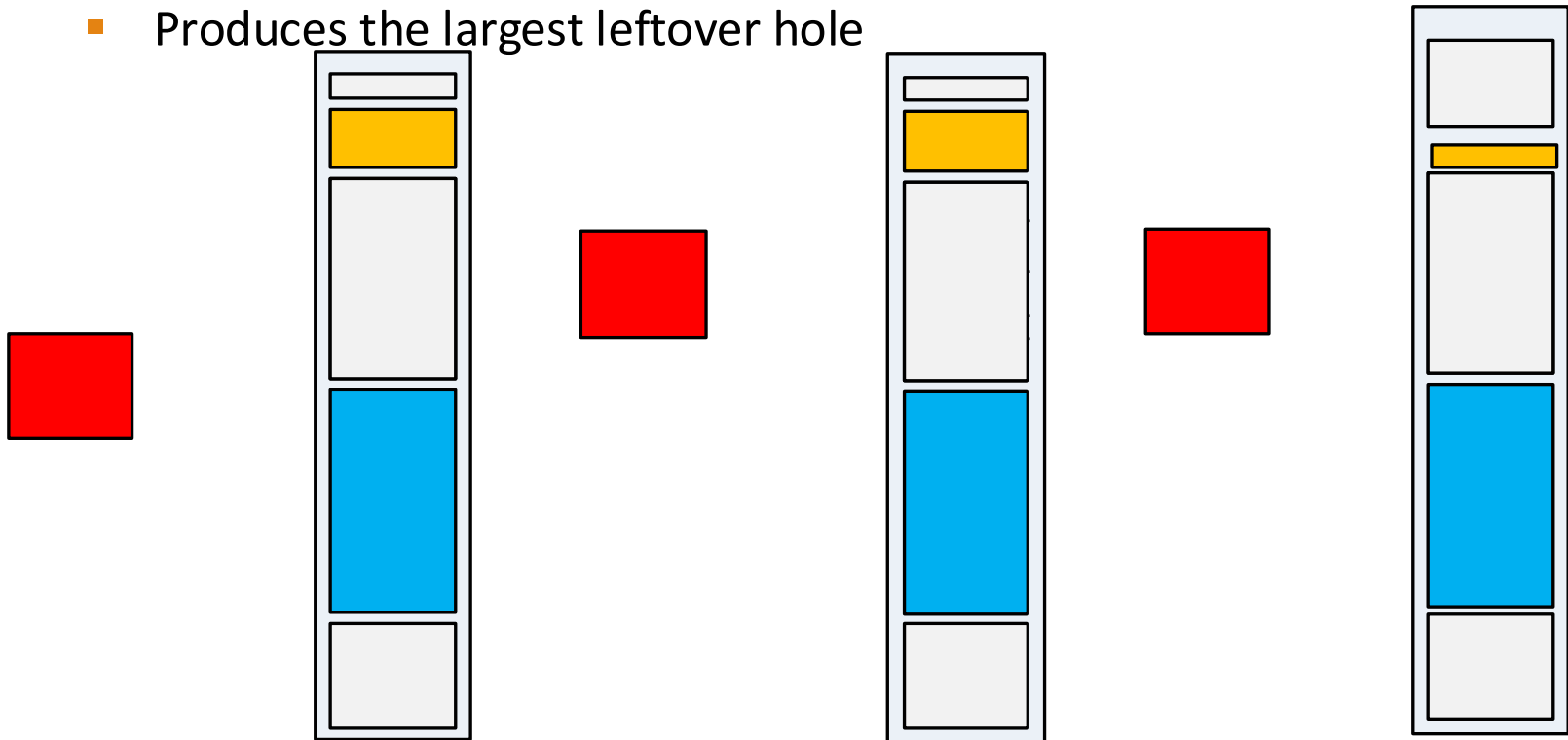
Partition Table

Memory Address	Size	Process	Usage
0x0	120k	4	In use
120k	60k		Free
180k	20k	5	In Use
200k	10k		Free



Contiguous Allocation...

- Dynamic Storage-Allocation Problem
 - **First-fit:** Allocate the first hole that is big enough
 - **Best-fit:** Allocate the smallest hole that is big enough; must search entire list, unless ordered by size
 - Produces the smallest leftover hole
 - **Worst-fit:** Allocate the largest hole; must also search entire list
 - Produces the largest leftover hole



Fragmentation

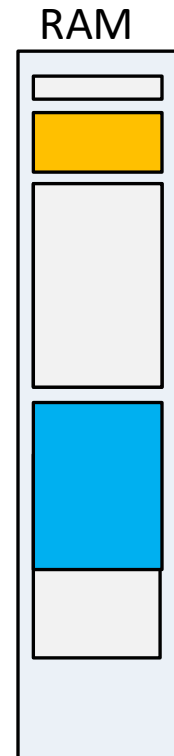
External Fragmentation – total memory space exists to satisfy a request, but it is not contiguous

Internal Fragmentation – allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used

Partition Table

Memory Address	Size	Process	Usage	
0x0	120k	4	In use	
120k	60k		Free	
180k	20k	5	In Use	
200k	10k		Free	

New process (6) of size 65k cannot start, even though 70k free memory is available



Fragmentation...

- Reduce external fragmentation by **compaction**
 - Shuffle memory contents to place all free memory together in one large block
 - Compaction is possible *only* if relocation is dynamic, and is done at execution time

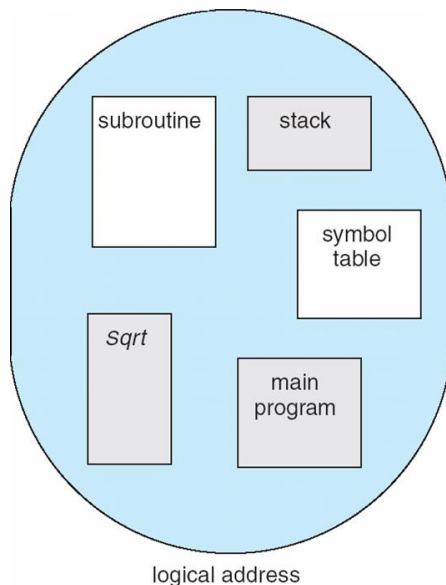
Segmentation

- Memory-management scheme that supports user view of memory

A program is a collection of segments

- A segment is a logical unit such as:

main program
procedure
function
method
object
local variables, global variables
common block
stack
symbol table
arrays



```
static unsigned int blk_flush_policy(unsigned int fflags, struct request *rq)
{
    unsigned int policy = 0;

    if (blk_rq_sectors(rq))
        policy |= REQ_FSEQ_DATA;

    return policy;
}

static unsigned int blk_flush_cur_seq(struct request *rq)
{
    return 1 << ffz(rq->flush.seq);
}

static void blk_flush_restore_request(struct request *rq)
{
    rq->bio = rq->biotail;
    rq->end_io = rq->flush.saved_end_io;
}

static bool blk_flush_queue_rq(struct request *rq, bool add_front)
{
    if (rq->q->mq_ops) {
        struct request_queue *q = rq->q;

        blk_mq_add_to_request_list(rq, add_front);
        blk_mq_kick_request_list(q);
        return false;
    }
}

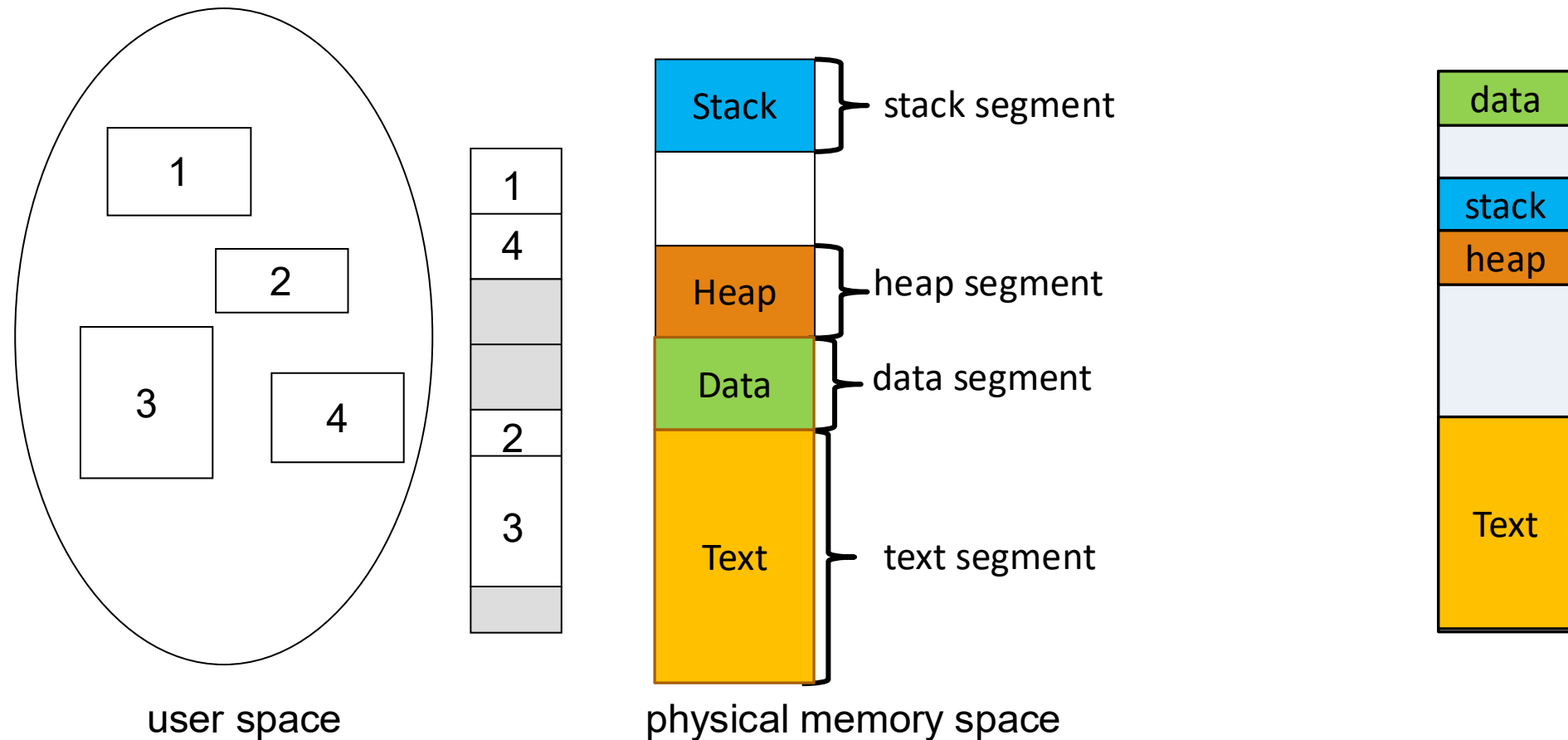
static bool blk_flush_complete_seq(struct request *rq,
    struct blk_flush_queue *fq,
    unsigned int seq, int error)
{
    struct request_queue *q = rq->q;
    struct list_head *pending = &fq->flush_queue[fq->flush_pending_idx];
    bool queued = false, kicked;

    BUG_ON(rq->flush.seq & seq);
    rq->flush.seq |= seq;
}

static void flush_end_io(struct request *flush_rq, int error)
{
    struct request_queue *q = flush_rq->q;
    struct list_head *running;
    bool queued = false;
    struct request *rq, *n;
    unsigned long flags = 0;
```

User's View of a Program

Commonly Used Segments (an example)



Segmentation Architecture

- Logical address consists of a two tuple:

$\langle \text{segment-number}, \text{offset} \rangle$,

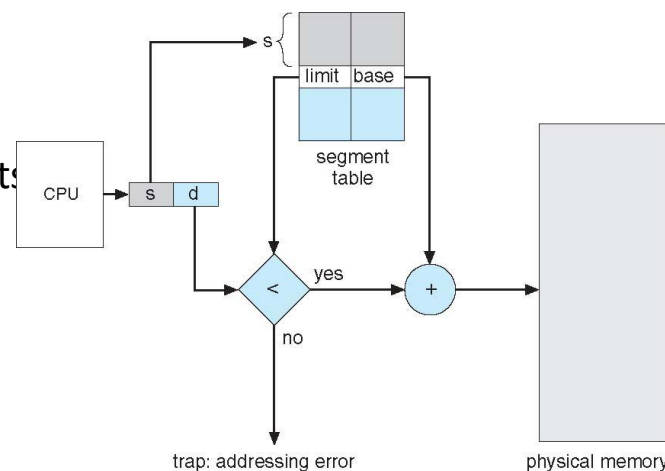
- **Segment table** – maps two-dimensional physical addresses; each table entry has:

- **base** – contains the starting physical address where the segments reside in memory
- **limit** – specifies the length of the segment

- **Segment-table base register (STBR)** points to the segment table's location in memory

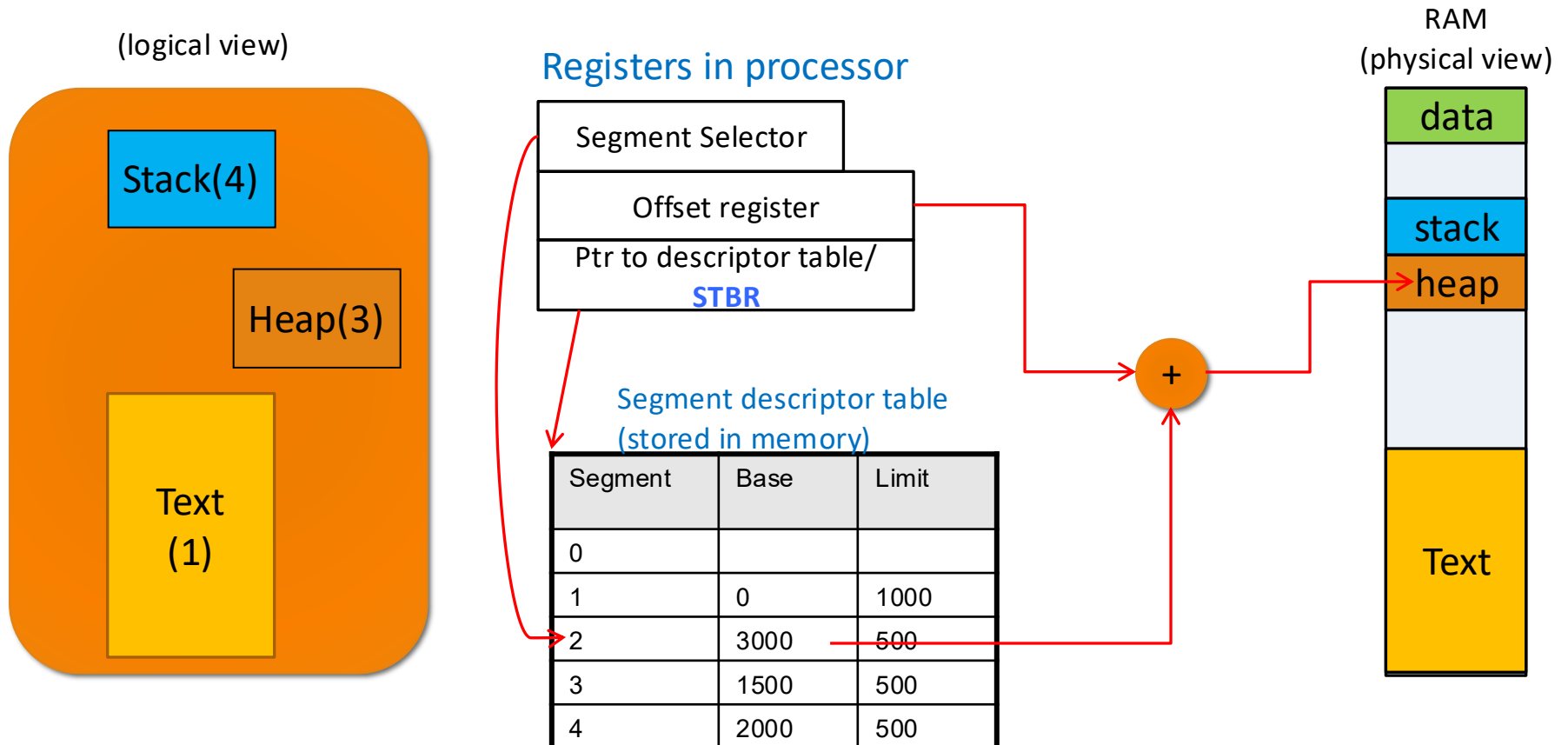
- **Segment-table length register (STLR)** indicates number of segments used by a program;

- segment number **s** is legal if **s** < **STLR**

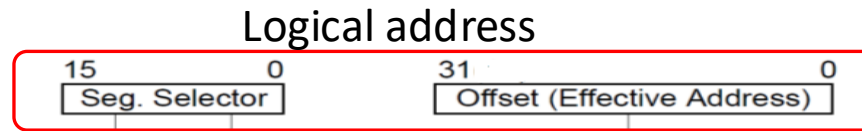
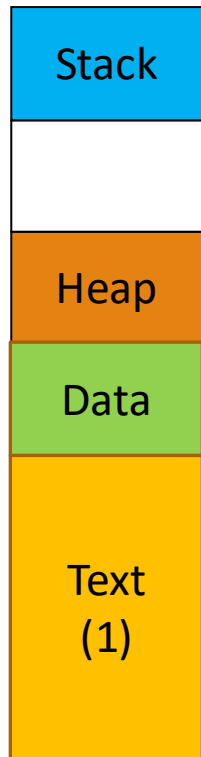


Segmentation Hardware

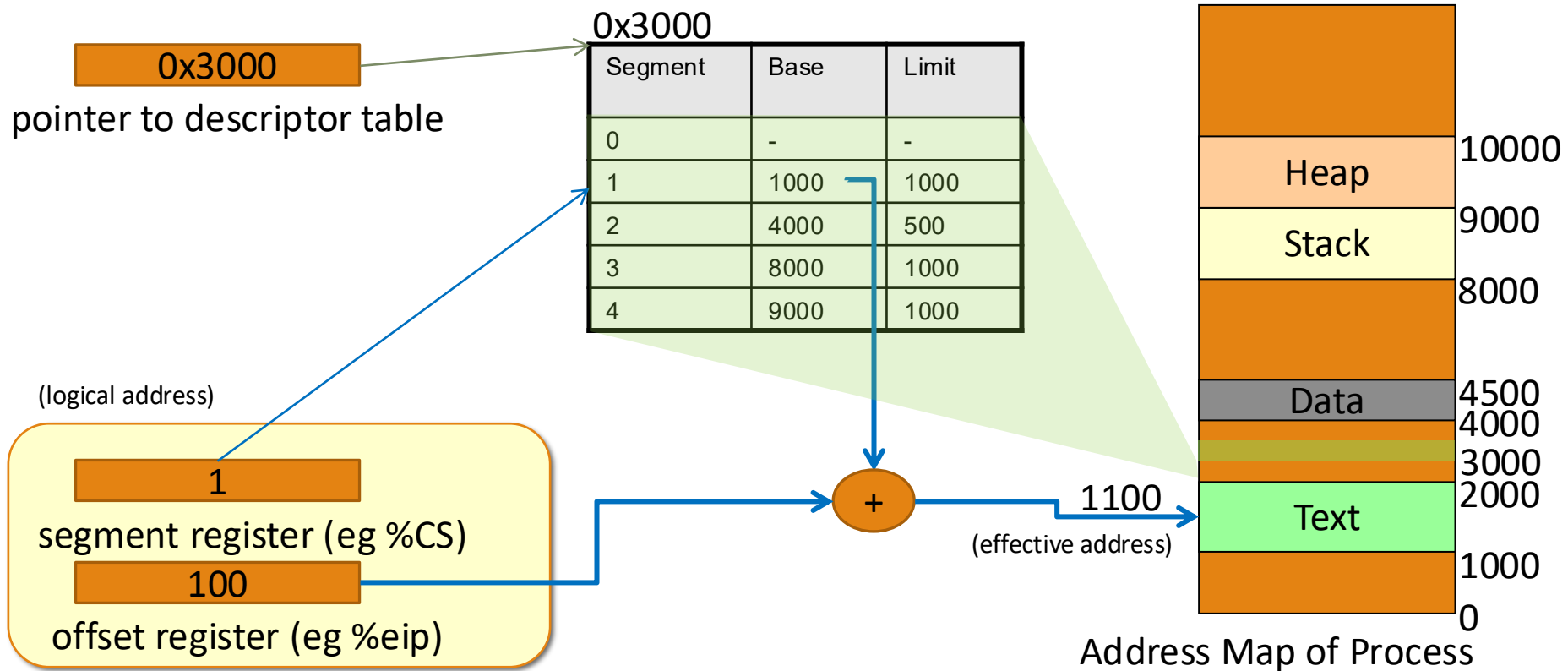
Segmentation Architecture ...



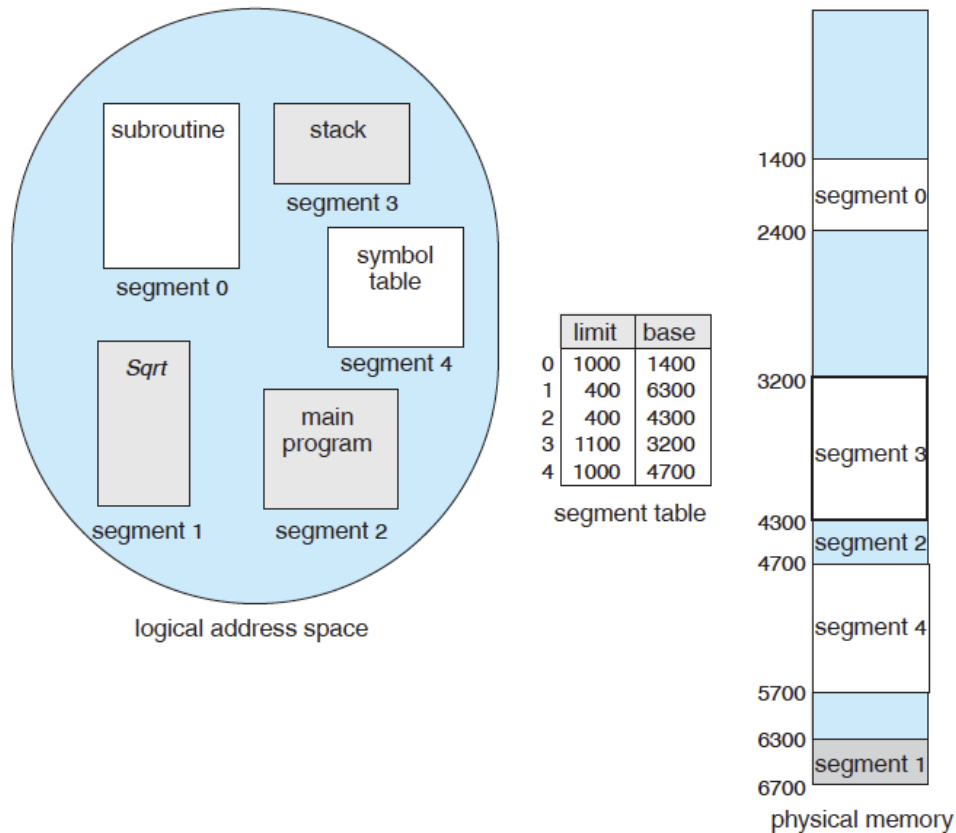
Segmentation (*logical* to *linear* address)



Example



Example of segmentation



Qn. mein LIMIT bhi de toh wo bhi LENGTH hi h

Example

Which of the following logical address will produce trap addressing error?

1. 0, 430
2. 1, 11
3. 2, 100
4. 3, 425
5. 4, 95

Calculate the physical address if no trap is produced.

Segment No.	Base	Length
0	1219	700
1	2300	14
2	90	100
3	1327	580
4	1952	96

Paging

- Physical address space of a process can be noncontiguous; process is allocated physical memory whenever the latter is available
 - Avoids external fragmentation
 - Avoids problem of varying sized memory chunks
- Divide physical memory into fixed-sized blocks called **frames**
 - Size is power of 2, between 512 bytes and 16 Mbytes
- Divide logical memory into blocks of same size called **pages**
- Keep track of all free frames
- To run a program of size ***N*** pages, need to find ***N*** free frames and load program
- Set up a **page table** to translate logical to physical addresses
- Backing store likewise split into pages
- Still have Internal fragmentation

Paging...

BLOCK = PAGE (interchangeably used)

RAM	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	

RAM is split into fixed size partitions called page frames.

Page frames typically 4KBs

process

1	1
2	2
3	3
4	4
5	5
6	6

Process split into blocks of equal size.

block size = page frame size

Per process page table (stored in RAM)

block	page frame
1	1
2	2
3	3
4	4
5	5
6	6

Paging...

RAM	
1	1
2	2
3	3
4	4
5	5
6	6
7	
8	
9	
10	
11	
12	
13	
14	

process1	
1	
2	
3	
4	
5	
6	

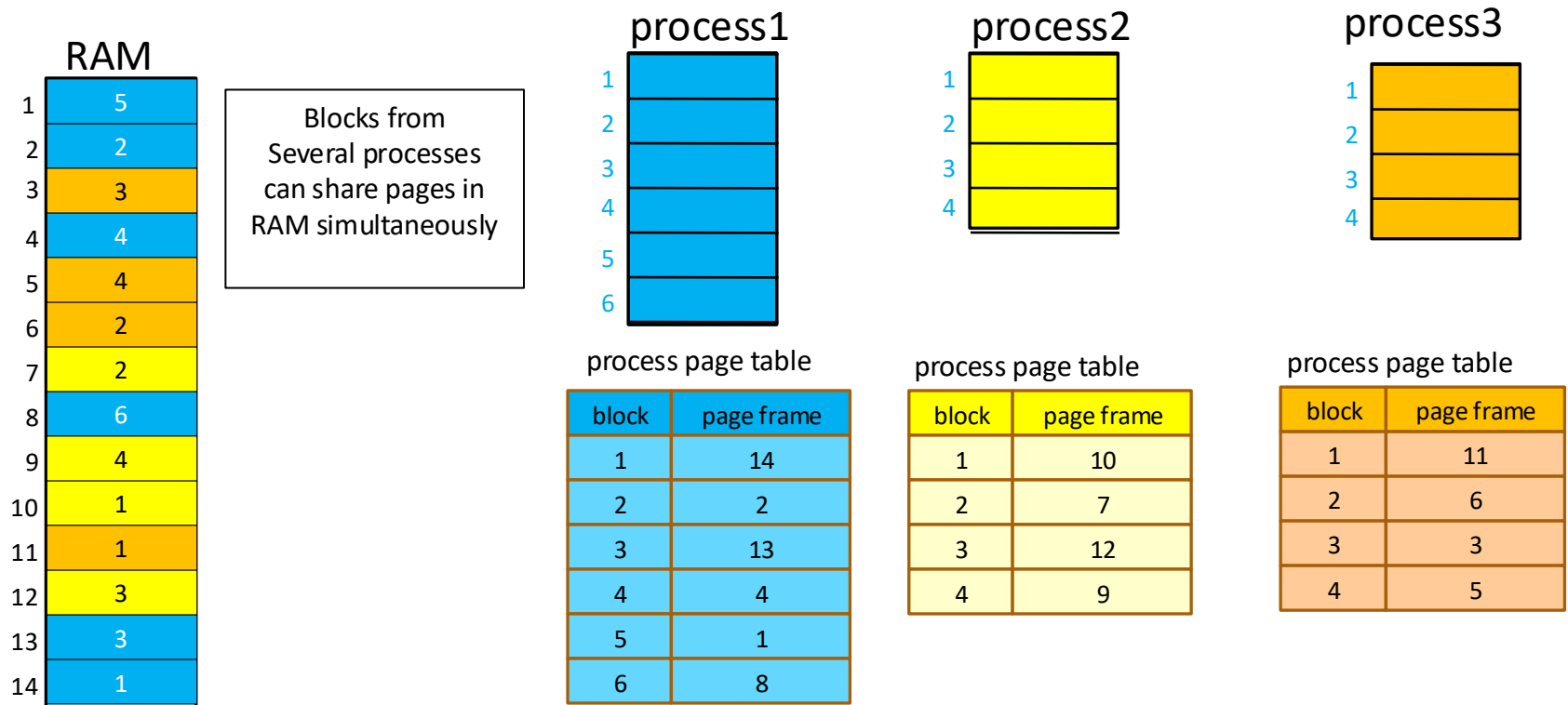
process page table

block	page frame
1	1
2	2
3	3
4	4
5	5
6	6

Because of the page table, blocks need not be in contiguous page frames

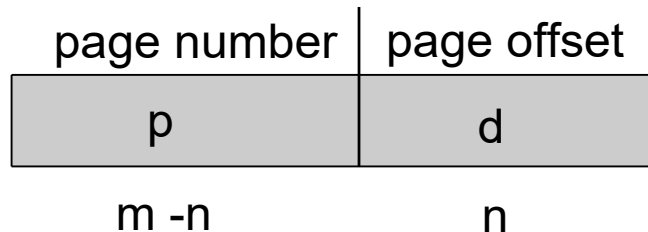
Every time a memory location is accessed, the processor looks into the page table to identify the corresponding page frame number.

Paging



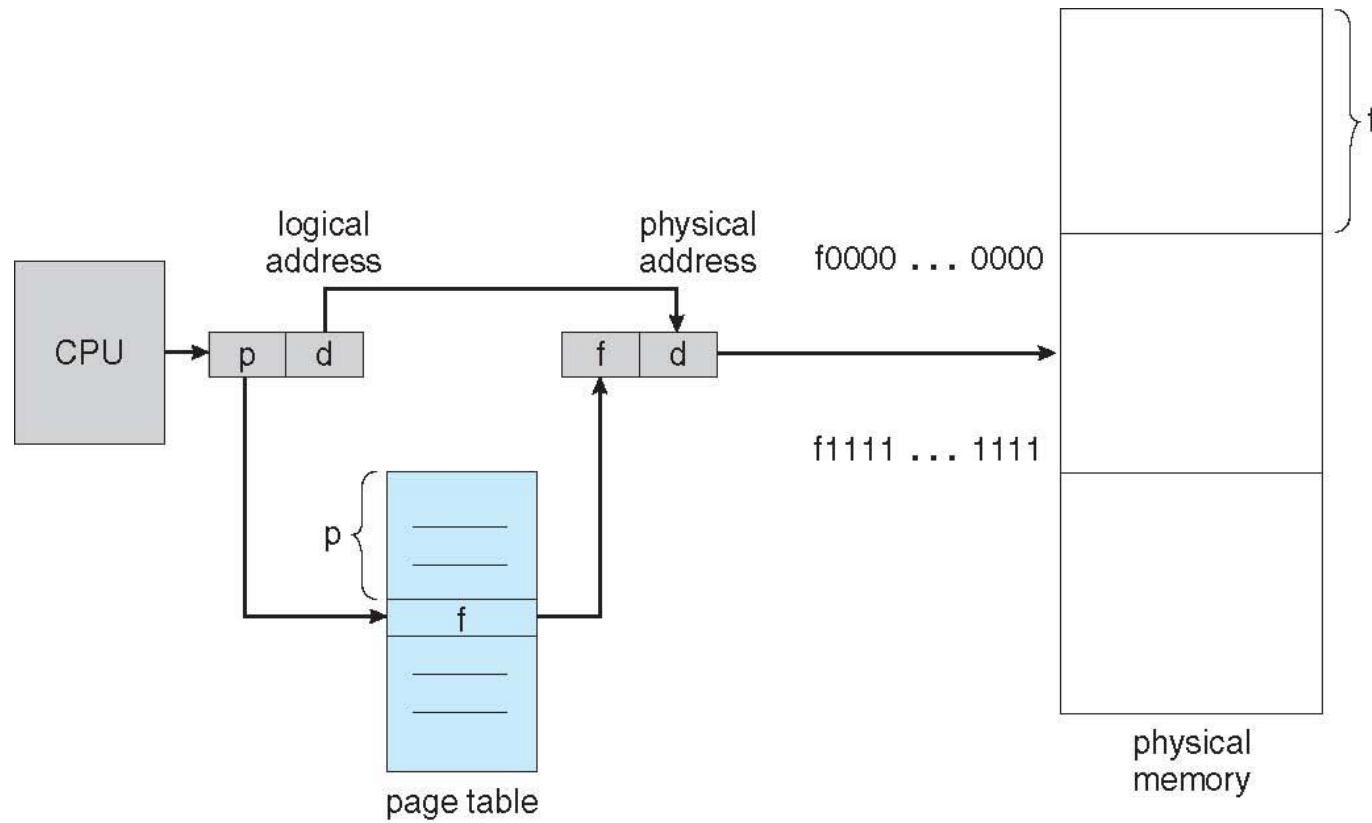
Address Translation Scheme

- Address generated by CPU is divided into:
 - **Page number (p)** – used as an index into a **page table** which contains base address of each page in physical memory
 - **Page offset (d)** – combined with base address to define the physical memory address that is sent to the memory unit

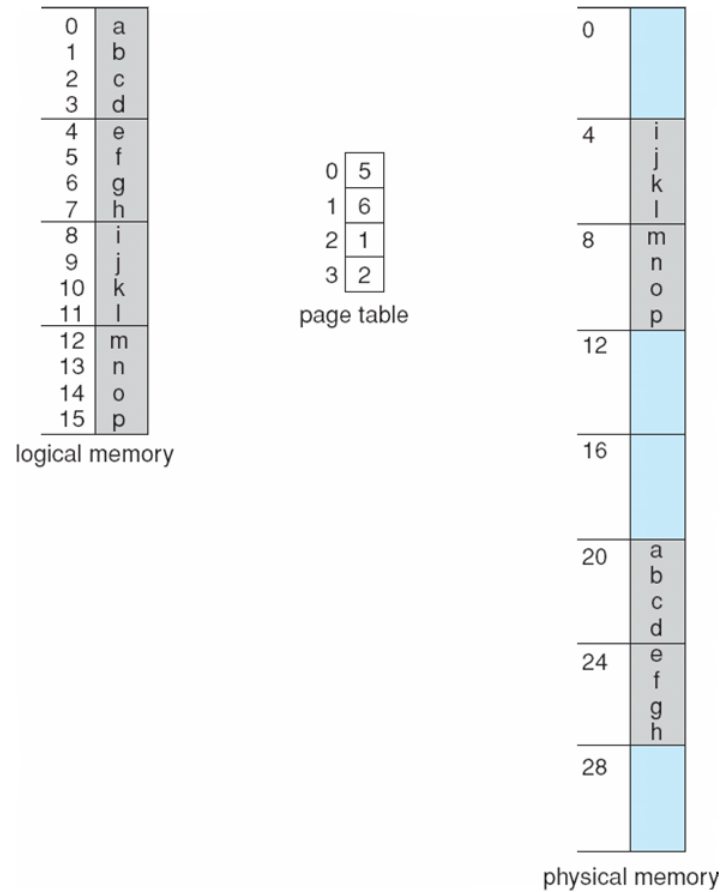


- For given logical address space 2^m and page size 2^n

Paging Hardware



Paging Example



$n=2$ and $m=4$ 32-byte memory and 4-byte pages

1 KB = 2^{10} bytes
1 MB = 2^{20} bytes

Consider a system with byte-addressable memory, 32 bit logical addresses, 4 kilobyte page size and page table entries of 4 bytes each. The size of the page table in the system in megabytes is 4.

Consider a machine with 64 MB physical memory and a 32 bit virtual address space. If the page size is 4 KB, what is the approximate size of the page table?

16 MB

Number of virtual pages = $2^{32} / 2^{12} = 2^{20}$

8 MB

Size of each page table entry (PTE):
Each entry must store a frame number.
Number of frames = $64\text{MB} / 4\text{KB} = 2^{14}$

✓ 2 MB

So, bits required for frame number = 14 bits $\approx$ 2 bytes (rounded)
In practice, we approximate PTE $\approx$ 2 bytes

24 MB

Page table size = $2^{20} \times 2 \text{ bytes} = 2^{21} \text{ bytes}$

Consider a memory management system that uses a page size of 2 KB. Assume that both the physical and virtual addresses start from 0. Assume that the pages 0,1,2 and 3 are stored in the page frames 1,3,2 and 0, respectively. The physical address (in decimal format) corresponding to the virtual address 2500 (in decimal format) is _____

Virtual address → Page 1, Offset 452

Page table → Page 1 maps to Frame 3

Physical address → Frame 3 + Offset 452

☒ Logical Address (Virtual Address)

Generated by CPU

Refers to a PAGE number + OFFSET

☞ It talks about pages, NOT frames

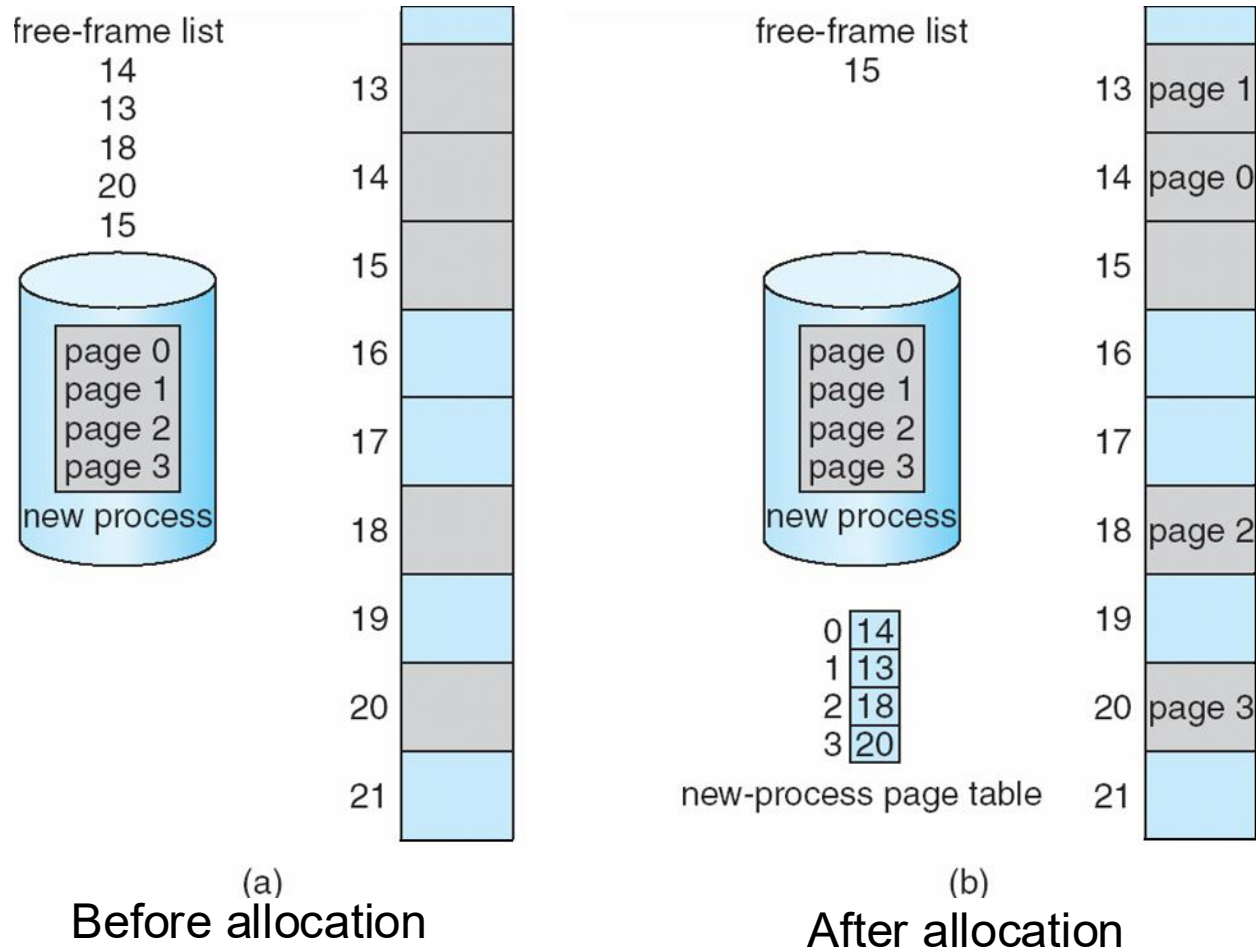
☒ Physical Address

Actual address in RAM

Refers to a FRAME number + OFFSET

Jis order mein free frame list h usi order mein page bhi insert ho raha h like in free-frame list it is 14 then 13 so in allocation first page 0 was put at 14 then page 1 at 13

Free Frames



Implementation of Page Table

- Page table is kept in main memory Along with other frames this table is also in a frame
- **Page-table base register (PTBR)** points to the page table
- **Page-table length register (PTLR)** indicates size of the page table
- In this scheme every data/instruction access requires two memory accesses
 - One for the page table and one for the data / instruction
- ✓ The two memory access problem can be solved by the use of a special fast-lookup hardware cache called **associative memory** or **translation look-aside buffers (TLBs)**

Implementation of Page Table...

- Some TLBs store **address-space identifiers (ASIDs)** in each TLB entry
 - uniquely identifies each process to provide address-space protection for that process
 - Otherwise need to flush at every context switch
- TLBs typically small (64 to 1,024 entries)
- On a TLB miss, value is loaded into the TLB for faster access next time
 - Replacement policies must be considered
- ✓ ○ Some entries can be **wired down** for permanent fast access

Associative Memory

Associative memory – parallel search

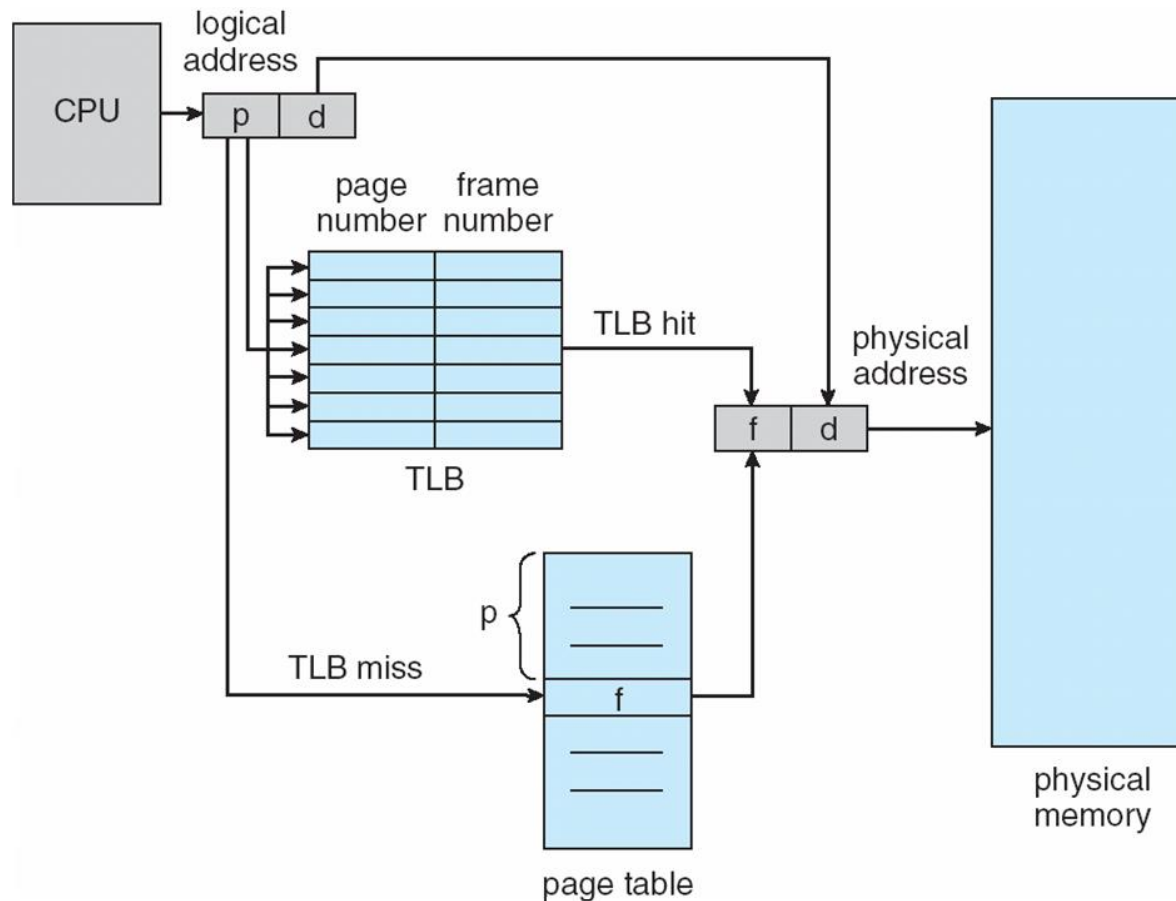
Page #	Frame #

TLB is implemented using associative memory

Address translation (p, d)

- If p is in associative register, get frame # out hit
- Otherwise get frame # from page table in memory miss

Paging Hardware With TLB



Effective Access Time

Associative Lookup = ε time unit

- Can be < 10% of memory access time

Hit ratio = α

- Hit ratio – percentage of times that a page number is found in the associative registers; ratio related to number of associative registers

Consider $\alpha = 80\%$, $\varepsilon = 20\text{ns}$ for TLB search, 100ns for memory access

Effective Access Time (EAT)

$$\text{EAT} := \text{TLB_miss_time} * (1 - \alpha) + \text{TLB_hit_time} * \alpha$$

$$\text{EAT} := (\varepsilon + 2 * \text{memory_access_time}) * (1 - \alpha) + (\varepsilon + \text{memory_access_time}) * \alpha$$

↳ For miss first it will also traverse TLB so thats why E

Consider $\alpha = 80\%$, $\varepsilon = 20\text{ns}$ for TLB search, 100ns for memory access

- $\text{EAT} = 0.80 \times 120 + 0.20 \times 220 = 140$

Example

Consider a paging hardware with a TLB. Assume that the entire page table and all the pages are in the physical memory. It takes 10 milliseconds to search the TLB and 80 milliseconds to access the physical memory. If the TLB hit ratio is 0.6. What is EAT (in milliseconds)?

122 ms

(Dont forget to include E while calculating time when missed as $(1 - \text{hit ratio}) * (E + 2 * T_m)$)

Consider a single level paging scheme with a TLB. Assume no page fault occurs. It takes 20 ns to search the TLB and 100 ns to access the physical memory. If TLB hit ratio is 80%, the effective memory access time is 0.00014 msec.

$$EAT = h \times (TLB + Memory) + (1 - h) \times (TLB + 2 \times Memory)$$

$$= 0.8 \times (20 + 100) + 0.2 \times (20 + 2 \times 100)$$

Consider a single level paging scheme with a TLB. Assume no page fault occurs. It takes 20 ns to search the TLB and 100 ns to access the physical memory. If effective memory access time is 130 ns, TLB hit ratio is 0.9.

Memory Protection

- Memory protection implemented by associating protection bit with each frame to indicate if read-only or read-write access is allowed
 - Can also add more bits to indicate page execute-only, and so on
- **Valid-invalid** bit attached to each entry in the page table:
 - “valid” indicates that the associated page is in the process’ logical address space, and is thus a legal page
 - “invalid” indicates that the page is not in the process’ logical address space
 - Or use **page-table length register (PTLR)**
- Any violations result in a trap to the kernel

00000	page 0
	page 1
	page 2
	page 3
	page 4
10,468	page 5
12,287	

frame number		valid-invalid bit	
0	2	v	
1	3	v	
2	4	v	
3	7	v	
4	8	v	
5	9	v	
6	0	i	
7	0	i	

page table

0	
1	
2	page 0
3	page 1
4	page 2
5	
6	
7	page 3
8	page 4
9	page 5
	⋮
	page n

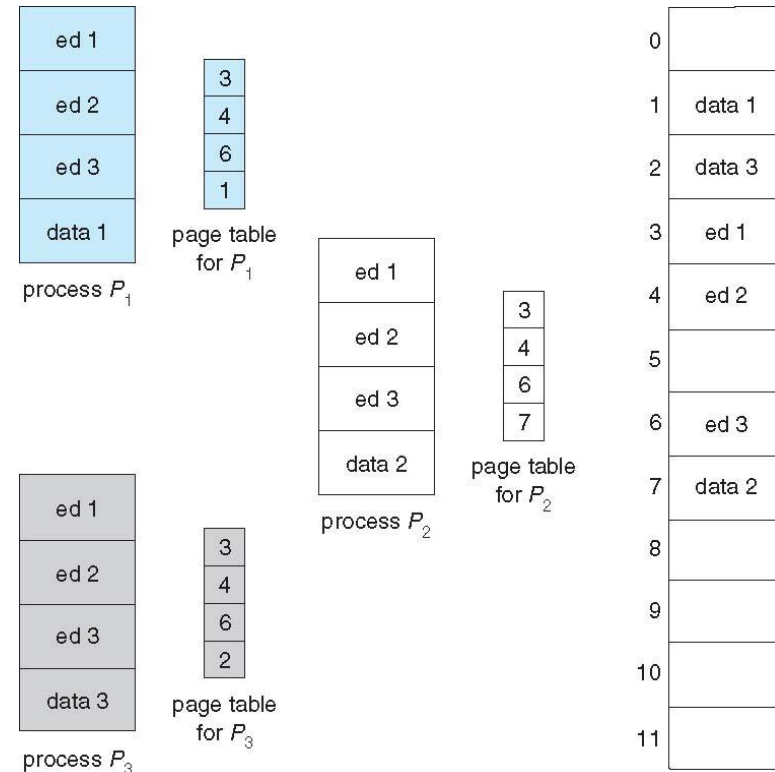
Shared Pages

Shared code

- One copy of read-only (**reentrant**) code shared among processes (i.e., text editors, compilers, window systems)
- Similar to multiple threads sharing the same process space
- Also useful for interprocess communication if sharing of read-write pages is allowed

Private code and data

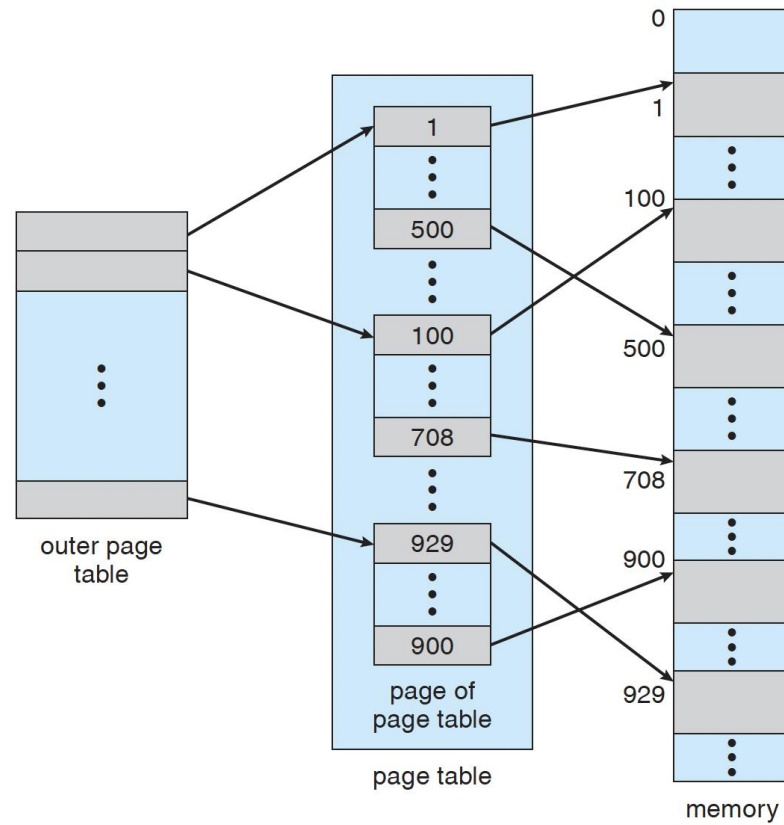
- Each process keeps a separate copy of the code and data
- The pages for the private code and data can appear anywhere in the logical address space



Structure of the Page Table

- Memory structures for paging can get huge using straight-forward methods
 - Consider a 32-bit logical address space as on modern computers
 - Page size of 4 KB (2^{12})
 - Page table would have 1 million entries ($2^{32} / 2^{12}$)
 - If each entry is 4 bytes -> 4 MB of physical address space / memory for page table alone
 - That amount of memory used to cost a lot
 - Don't want to allocate that contiguously in main memory
 - Hierarchical Paging
 - Hashed Page Tables
 - Inverted Page Tables
- Solutions to reduce cost of storing whole page table in the main memory

A two-level page-table scheme



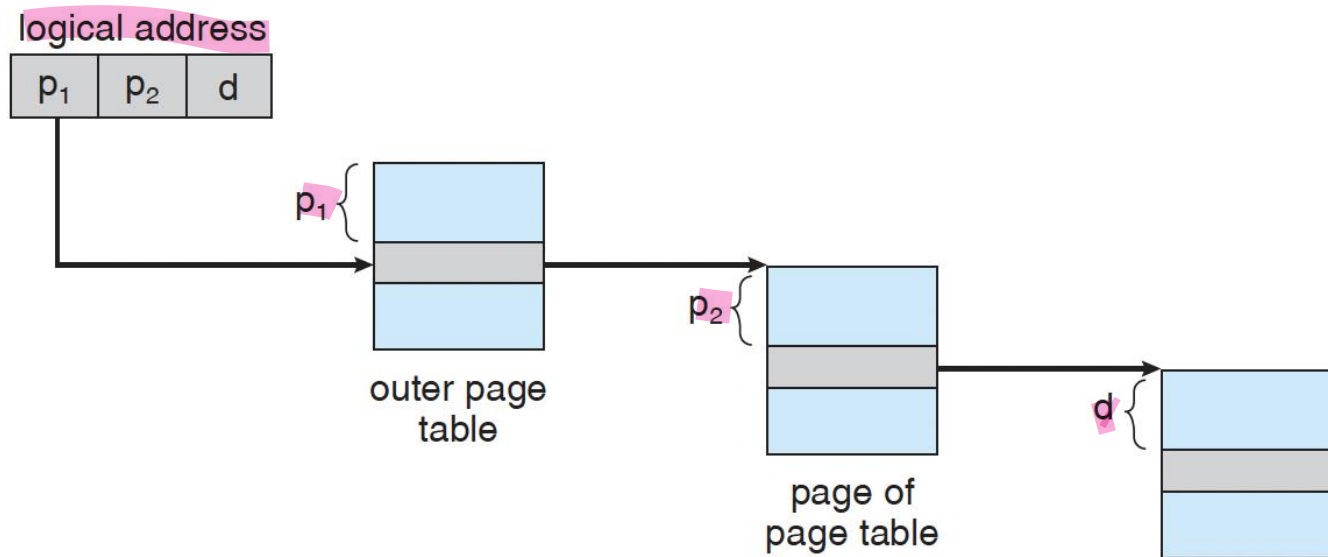


Figure 7.15 Address translation for a two-level 32-bit paging architecture.

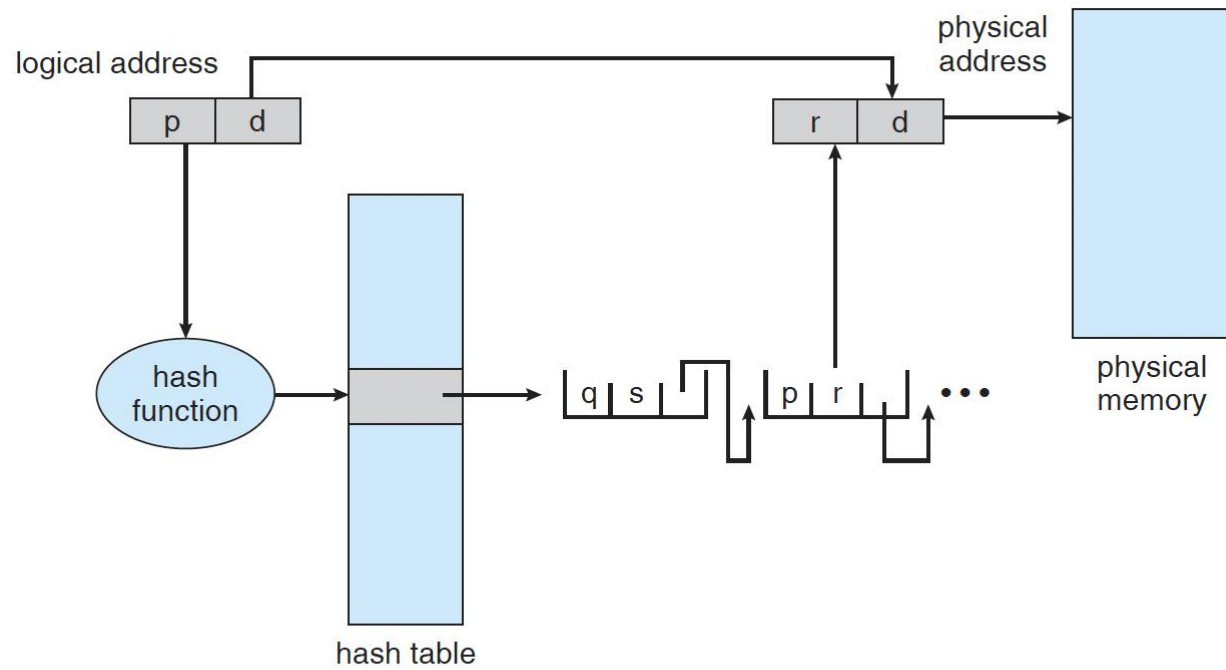


Figure 7.16 Hashed page table.